

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S. Bhuvanesh	Reg. No.:	192521004
Section / Batch:	IT	Date:	20/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I BHUVANESH..S (192521004)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: BHUVANESH.S

Assigned: Q5. Optimal Binary Search Tree (OBST)

A database frequently searches for certain keys, each having a known access frequency.

Constructing a Binary

Search Tree without considering frequencies may lead to inefficient searches. The goal is to build an Optimal Binary

Search Tree that minimizes the expected search cost using Dynamic Programming.

Sample Input:

$n = 3$

Keys = 10 12 20

Freq = 34 8 50

Expected Output:

Minimum Cost = 142

1. Aim

To construct an Optimal Binary Search Tree (OBST) using Dynamic Programming such that the expected search cost is minimized based on the access frequencies of the keys.

2. Problem Statement

Given a set of sorted keys and their corresponding search frequencies, construct a Binary Search Tree in which frequently searched keys are placed closer to the root. The objective is to find the BST with the minimum possible search cost.

3. Algorithm

- Read the number of keys n , keys, and their frequencies.
- Create a cost table $\text{cost}[i][j]$.
- For a single key, set its cost equal to its frequency.
- Consider chains of length 2, 3, ..., n .
- For each range of keys, try every key as the root.
- Calculate the cost of left subtree + right subtree + total frequency.
- Store the minimum cost in $\text{cost}[i][j]$.
- The value $\text{cost}[0][n-1]$ is the minimum OBST cost.

4. Input / Output Format

Input:

$n = 3$

Keys = 10 12 20

Freq = 34 8 50

Output:

Minimum Cost = 142

5. Theory / Problem Idea

In a normal BST, all keys may not have the same search frequency. If a frequently searched key is placed deep in the tree, the total search cost increases.

OBST uses the frequencies to determine the best root and arrangement. Dynamic Programming avoids repeatedly solving the same subproblems.

The search cost is calculated as:

Frequency $\times$ Level

The goal is to minimize the total weighted search cost.

6. Recurrence Relation

Let $\text{cost}[i][j]$ represent the minimum cost for keys from index i to j .

For every possible root r :

Base case:

If there is no left or right subtree, its cost is considered 0.

main.py

```
1 def optimal_bst(freq):
2     n = len(freq)
3     cost = [[0] * n for _ in range(n)]
4
5     for i in range(n):
6         cost[i][i] = freq[i]
7
8     for length in range(2, n + 1):
9         for i in range(n - length + 1):
10             j = i + length - 1
11             total = sum(freq[i:j + 1])
12             cost[i][j] = float('inf')
13
14             for r in range(i, j + 1):
15                 left = cost[i][r - 1] if r > i else 0
16                 right = cost[r + 1][j] if r < j else 0
17                 cost[i][j] = min(cost[i][j], left + right + total)
18
19     return cost[0][n - 1]
20
21 n = 3
22 keys = [10, 12, 20]
23 freq = [34, 8, 50]
24
25 print("Minimum Cost =", optimal_bst(freq))
```

input

Minimum Cost = 142

7. Dry Run

Given:

Keys = 10, 12, 20

Freq = 34, 8, 50

Single keys:

Cost[10] = 34

Cost[12] = 8

Cost[20] = 50

Two keys:

For 10, 12:

- Root 10 $\rightarrow 8 + 42 = 50$
- Root 12 $\rightarrow 34 + 42 = 76$

Minimum = 50

For 12, 20:

- Root 12 $\rightarrow 50 + 58 = 108$
- Root 20 $\rightarrow 8 + 58 = 66$

Minimum = 66

Three keys:

Total frequency:

Try each root:

- Root 10: $0 + 66 + 92 = 158$
- Root 12: $34 + 50 + 92 = 176$
- Root 20: $50 + 0 + 92 = 142$

Therefore:

Minimum Cost = 142

The optimal root is 20.

8. Complexity Analysis

Complexity	Value
------------	-------

Time Complexity	$O(n^3)$
-----------------	----------

Space Complexity	$O(n^2)$
------------------	----------

The three nested considerations are used to examine key ranges and possible roots.

9. result:

For the given input:

$n = 3$

Keys = 10 12 20

Freq = 34 8 50

The Optimal Binary Search Tree selects **20 as the root**, because it has the highest search frequency.

The minimum search cost is:

Minimum Cost = 142

10. Conclusion

The Optimal Binary Search Tree successfully minimizes the expected search cost by placing frequently accessed keys closer to the root. Using Dynamic Programming, the minimum cost for the given input is 142, with key 20 as the optimal root.

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____